Resolución de Defensa Oral y Práctica (TP N° 5)

Punto 1: Tabla de Servicios Adicionales y Consumos

1.1 Script DDL para Crear las Tablas

```sql
-- 1. Tabla de catálogo de servicios con su precio actual:
CREATE TABLE IF NOT EXISTS servicio_adicional (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    precio_actual NUMERIC(10,2) NOT NULL CHECK (precio_actual >= 0)
);

-- 2. Tabla de consumos durante una reserva:
CREATE TABLE IF NOT EXISTS consumo_servicio (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_reserva BIGINT NOT NULL REFERENCES reserva(id) ON DELETE RESTRICT,
    id_servicio INT NOT NULL REFERENCES servicio_adicional(id) ON DELETE RESTRICT,
    cantidad INT NOT NULL,
    precio_unitario NUMERIC(10,2) NOT NULL,
    fecha_hora_consumo TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    -- Restricción: Impide cantidades menores o iguales a cero
    CONSTRAINT chk_consumo_cantidad CHECK (cantidad > 0),

    -- Restricción: Impide precios unitarios negativos
    CONSTRAINT chk_consumo_precio_unitario CHECK (precio_unitario >= 0)
);

-- 3. Índice para optimizar búsquedas de consumos por reserva
CREATE INDEX IF NOT EXISTS idx_consumo_servicio_id_reserva
ON consumo_servicio(id_reserva);
```

1.2 Justificación Técnica (Qué responder si pregunta)

- **Sobre el precio unitario guardado en el consumo:**

  > *"Ingeniero, se guarda el `precio_unitario` en la tabla transaccional `consumo_servicio` como una desnormalización controlada para capturar el valor histórico exacto al momento del consumo. Si el precio en el catálogo (`servicio_adicional`) cambia en el futuro, los consumos facturados no se modifican".*

- **Sobre las restricciones:**

  > *"Se agregaron dos `CHECK`: `cantidad > 0` para garantizar unidades válidas y `precio_unitario >= 0` para evitar valores monetarios negativos (permitiendo 0 si fuera una cortesía)".*

Punto 2: Inserción de 20 Consumos y Consultas de Verificación

2.1 Paso Previo: Catálogo y Preparación de Fechas (Para sincronizar con Punto 3)

```
-- 1. Cargar servicios en el catálogo si aún no están:
INSERT INTO servicio_adicional (nombre, precio_actual) VALUES
('Desayuno Buffet Premium', 4500.00),
('Almuerzo Ejecutivo', 9500.00),
('Cena Gourmet', 14000.00),
('Acceso Spa y Masajes', 12000.00),
('Frigobar y Snacks', 3500.00);

-- 2. Asegurar que las reservas 1 a 20 sean CONFIRMADAS y salgan entre Julio y
Diciembre 2026 (para que el Punto 3 devuelva datos):
UPDATE reserva
SET estado = 'CONFIRMADA',
    fecha_inicio = '2026-07-15',
    fecha_salida = '2026-07-25'
WHERE id BETWEEN 1 AND 20;
```

2.2 Inserción de 20 Consumos Válidos (Sin Claves Foráneas Inválidas)

```
INSERT INTO consumo_servicio (id_reserva, id_servicio, cantidad, precio_unitario,
fecha_hora_consumo) VALUES
(1,  1, 2, 4500.00, '2026-07-16 09:00:00'),
(2,  2, 1, 9500.00, '2026-07-16 13:30:00'),
(3,  3, 2, 14000.00, '2026-07-16 21:00:00'),
(4,  4, 1, 12000.00, '2026-07-17 16:00:00'),
(5,  5, 3, 3500.00, '2026-07-17 23:15:00'),
(6,  1, 2, 4500.00, '2026-07-18 08:45:00'),
(7,  2, 2, 9500.00, '2026-07-18 14:00:00'),
(8,  3, 1, 14000.00, '2026-07-18 21:30:00'),
(9,  4, 2, 12000.00, '2026-07-19 11:00:00'),
(10, 5, 1, 3500.00, '2026-07-19 19:20:00'),
(11, 1, 1, 4500.00, '2026-07-20 09:10:00'),
(12, 2, 3, 9500.00, '2026-07-20 13:45:00'),
(13, 3, 2, 14000.00, '2026-07-21 20:30:00'),
(14, 4, 1, 12000.00, '2026-07-21 17:00:00'),
(15, 5, 4, 3500.00, '2026-07-22 22:00:00'),
(16, 1, 2, 4500.00, '2026-07-22 09:30:00'),
(17, 2, 1, 9500.00, '2026-07-23 13:00:00'),
(18, 3, 2, 14000.00, '2026-07-23 21:15:00'),
(19, 4, 1, 12000.00, '2026-07-24 15:45:00'),
(20, 5, 2, 3500.00, '2026-07-24 23:00:00');
```

2.3 Consultas de Verificación Solicitadas

Verificación A: Cantidad de consumos registrados

```sql
SELECT COUNT(*) AS total_consumos_registrados
FROM consumo_servicio;
```

(Debe devolver 20 o más).

Verificación B: Que todas las reservas y servicios referenciados existan

```sql
-- 1. Búsqueda de consumos con claves foráneas huérfanas (debe dar 0 filas):
SELECT cs.id AS id_consumo, cs.id_reserva, cs.id_servicio
FROM consumo_servicio cs
LEFT JOIN reserva r ON cs.id_reserva = r.id
LEFT JOIN servicio_adicional sa ON cs.id_servicio = sa.id
WHERE r.id IS NULL OR sa.id IS NULL;

-- 2. Resumen positivo de coincidencia total:
SELECT
    COUNT(cs.id) AS total_consumos,
    COUNT(r.id)  AS reservas_existentes_validas,
    COUNT(sa.id) AS servicios_existentes_validos
FROM consumo_servicio cs
LEFT JOIN reserva r ON cs.id_reserva = r.id
LEFT JOIN servicio_adicional sa ON cs.id_servicio = sa.id;
```

Verificación C: Que no existan cantidades inválidas (<= 0) ni precios negativos (< 0)

```sql
-- 1. Registros inválidos (debe dar 0 filas):
SELECT *
FROM consumo_servicio
WHERE cantidad <= 0 OR precio_unitario < 0;

-- 2. Resumen de rangos mínimos y máximos:
SELECT
    MIN(cantidad) AS cantidad_minima,
    MIN(precio_unitario) AS precio_minimo,
    MAX(cantidad) AS cantidad_maxima,
    MAX(precio_unitario) AS precio_maximo
FROM consumo_servicio;
```

Punto 3: Consumos por Hotel (Reservas Confirmadas 01/07/2026 al 31/12/2026)

> **Aclaración sobre la corrección del docente:** Al quitar "*agrupados por hotel*" del inicio del enunciado ("*Mostrar por hotel los consumos...*"), la consulta principal presenta el **detalle de cada consumo individual** mostrando sus datos de hotel, habitación, huésped y fechas, ordenado por hotel.

3.1 Consulta Principal (Detalle por Hotel, Fila a Fila)

```
SELECT
    h.nombre AS hotel,
    hab.numero AS numero_habitacion,
    CASE
        WHEN hab.capacidad = 1 THEN 'Individual'
        WHEN hab.capacidad = 2 THEN 'Doble'
        WHEN hab.capacidad = 3 THEN 'Triple'
        ELSE 'Suite Familiar'
    END AS tipo_habitacion,
    COALESCE(com.comodidades, 'Sin comodidades registradas') AS comodidades,
    COALESCE(pts.puntos_interes, 'Sin puntos de interés') AS punto_interes,
    (cli.nombre || ' ' || cli.apellido) AS huesped_titular,
    (r.fecha_inicio - INTERVAL '15 days')::DATE AS fecha_reserva,
    r.fecha_inicio AS fecha_ingreso,
    r.fecha_salida AS fecha_salida,
    sa.nombre AS servicio_consumido,
    cs.cantidad AS cantidad_consumos_registrados,
    (cs.cantidad * cs.precio_unitario) AS ingreso_total_generado
FROM consumo_servicio cs
INNER JOIN reserva r ON r.id = cs.id_reserva
INNER JOIN habitacion hab ON hab.id = r.id_habitacion
INNER JOIN hotel h ON h.id = hab.hotel_id
INNER JOIN huesped cli ON cli.id = r.id_huesped
INNER JOIN servicio_adicional sa ON sa.id = cs.id_servicio

-- Subconsultas para evitar producto cartesiano por relaciones N:M:
LEFT JOIN (
    SELECT hc.id_habitacion, STRING_AGG(DISTINCT c.nombre, ', ') AS comodidades
    FROM habitacion_comodidad hc
    INNER JOIN comodidad c ON c.id = hc.id_comodidad
    GROUP BY hc.id_habitacion
) com ON com.id_habitacion = hab.id

LEFT JOIN (
    SELECT hpi.id_hotel, STRING_AGG(DISTINCT pi.nombre, ', ') AS puntos_interes
    FROM hotel_punto_interes hpi
    INNER JOIN punto_interes pi ON pi.id = hpi.id_punto_interes
    GROUP BY hpi.id_hotel
) pts ON pts.id_hotel = h.id

WHERE r.estado = 'CONFIRMADA'
```

```
  AND r.fecha_salida BETWEEN '2026-07-01' AND '2026-12-31'

ORDER BY
    h.nombre ASC,
    ingreso_total_generado DESC;
```

3.2 Consulta Alternativa (Con Agrupación si lo solicita)

```
SELECT
    h.nombre AS hotel,
    hab.numero AS numero_habitacion,
    CASE
        WHEN hab.capacidad = 1 THEN 'Individual'
        WHEN hab.capacidad = 2 THEN 'Doble'
        WHEN hab.capacidad = 3 THEN 'Triple'
        ELSE 'Suite Familiar'
    END AS tipo_habitacion,
    COALESCE(com.comodidades, 'Sin comodidades registradas') AS comodidades,
    COALESCE(pts.puntos_interes, 'Sin puntos de interés') AS punto_interes,
    (cli.nombre || ' ' || cli.apellido) AS huesped_titular,
    (r.fecha_inicio - INTERVAL '15 days')::DATE AS fecha_reserva,
    r.fecha_inicio AS fecha_ingreso,
    r.fecha_salida AS fecha_salida,
    sa.nombre AS servicio_consumido,
    SUM(cs.cantidad) AS cantidad_consumos_registrados,
    SUM(cs.cantidad * cs.precio_unitario) AS ingreso_total_generado
FROM consumo_servicio cs
INNER JOIN reserva r ON r.id = cs.id_reserva
INNER JOIN habitacion hab ON hab.id = r.id_habitacion
INNER JOIN hotel h ON h.id = hab.hotel_id
INNER JOIN huesped cli ON cli.id = r.id_huesped
INNER JOIN servicio_adicional sa ON sa.id = cs.id_servicio
LEFT JOIN (
    SELECT hc.id_habitacion, STRING_AGG(DISTINCT c.nombre, ', ') AS comodidades
    FROM habitacion_comodidad hc
    INNER JOIN comodidad c ON c.id = hc.id_comodidad
    GROUP BY hc.id_habitacion
) com ON com.id_habitacion = hab.id
LEFT JOIN (
    SELECT hpi.id_hotel, STRING_AGG(DISTINCT pi.nombre, ', ') AS puntos_interes
    FROM hotel_punto_interes hpi
    INNER JOIN punto_interes pi ON pi.id = hpi.id_punto_interes
    GROUP BY hpi.id_hotel
) pts ON pts.id_hotel = h.id
WHERE r.estado = 'CONFIRMADA'
  AND r.fecha_salida BETWEEN '2026-07-01' AND '2026-12-31'
GROUP BY
    h.id, h.nombre, hab.numero, hab.capacidad, com.comodidades,
    pts.puntos_interes, cli.nombre, cli.apellido, r.fecha_inicio, r.fecha_salida,
sa.nombre
ORDER BY ingreso_total_generado DESC, h.nombre ASC;
```

Punto 4: EXPLAIN, Creación de Índice y Justificación Teórica

4.1 Consulta Objetivo (Reservas Confirmadas - Segundo Semestre 2026)

```
SELECT id, id_huesped, id_habitacion, fecha_inicio, fecha_salida, estado, tarifa
FROM reserva
WHERE estado = 'CONFIRMADA'
  AND fecha_salida BETWEEN '2026-07-01' AND '2026-12-31';
```

4.2 Paso 1: Ejecutar EXPLAIN (Antes del Índice)

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, id_huesped, id_habitacion, fecha_inicio, fecha_salida, estado, tarifa
FROM reserva
WHERE estado = 'CONFIRMADA'
  AND fecha_salida BETWEEN '2026-07-01' AND '2026-12-31';
```

- **Diagnóstico del Plan:** Realiza un `Seq Scan` recorriendo los 6 millones de registros de la tabla con un filtro en caliente (`Filter: ((estado = 'CONFIRMADA') AND (fecha_salida >= '2026-07-01') AND (fecha_salida <= '2026-12-31'))`), requiriendo cientos de miles de buffers de memoria compartida y varios segundos de CPU.

4.3 Paso 2: Crear el Índice Adecuado

El índice idóneo es un B-Tree compuesto respetando la columna de igualdad al inicio y la de rango en segundo lugar:

```
CREATE INDEX idx_reserva_estado_fecha_salida
ON reserva (estado, fecha_salida);
```

4.4 Paso 3: Ejecutar EXPLAIN (Después del Índice)

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, id_huesped, id_habitacion, fecha_inicio, fecha_salida, estado, tarifa
FROM reserva
WHERE estado = 'CONFIRMADA'
  AND fecha_salida BETWEEN '2026-07-01' AND '2026-12-31';
```

- **Diagnóstico del Plan Optimizado:** El planificador ahora utiliza `Bitmap Index Scan` o `Index Scan` sobre `idx_reserva_estado_fecha_salida` con condición `Index Cond: ((estado = 'CONFIRMADA') AND (fecha_salida >= '2026-07-01') AND (fecha_salida <= '2026-12-31'))`. El tiempo de respuesta cae a pocos milisegundos y el costo I/O se reduce al mínimo.

4.5 Respuestas Didácticas para la Defensa

1. **Por qué se eligió ese orden de columnas (`estado, fecha_salida`):**

   > *"Ingeniero, se aplica la **Regla de Oro de los Índices Compuestos: Igualdad antes que Rango**. La columna con filtro exacto de igualdad (`estado = 'CONFIRMADA'`) debe ir en primer lugar, y la columna con filtro de rango (`fecha_salida BETWEEN ...`) en segundo lugar. Si colocáramos primero la fecha, el árbol B-Tree saltaría a las fechas pero tendría que evaluar el estado navegando de forma dispersa. Colocando `estado` primero, el motor desciende directamente a la rama exclusiva de reservas confirmadas y, dentro de ese subconjunto ya ordenado, recorre de forma continua el intervalo de fechas solicitado".*

2. **Cómo se relaciona el índice con los filtros de la consulta:**

   > *"El índice B-Tree almacena las claves ordenadas primero por `estado` y luego por `fecha_salida`, asociadas a los identificadores físicos de tupla (`TID`: Block Number + Offset). Ambos predicados del `WHERE` se convierten en condiciones directas de búsqueda (`Index Cond`). El motor evita leer secuencialmente los 6.000.000 de registros en el Heap y solo recupera de disco las páginas de datos que contienen tuplas coincidentes".*

3. **Por qué un índice NO garantiza automáticamente una mejora del rendimiento:**

   > *"Un índice no garantiza una mejora automática debido a tres factores fundamentales:*
   >
   > 1. **Baja selectividad:** Si la consulta recupera un porcentaje alto de filas de la tabla (por ejemplo, más del 15% al 20%), el costo de leer las páginas del índice sumado a los accesos aleatorios al Heap (`Random Page Cost`) resulta más caro que recorrer la tabla entera secuencialmente (`Seq Scan`). En ese caso, el planificador descartará el índice.*
   > 2. **Costo de mantenimiento en escrituras (Overhead):** Cada índice penaliza las operaciones de `INSERT`, `UPDATE` y `DELETE`, ya que el motor debe mantener actualizadas las ramas del árbol B-Tree en disco en cada transacción.*
   > 3. **Tablas pequeñas o memoria caché:** Si una tabla tiene pocas páginas, es mucho más veloz leerla completa en memoria RAM mediante un escaneo secuencial que realizar la navegación por los nodos del árbol B-Tree"*.

Punto 5: Configuración de `max_connections = 120` en MySQL / MariaDB

5.1 Identificar el archivo de configuración utilizado por el servicio

- **En Linux / Contenedores Docker:**
  - Rutas estándar: `/etc/mysql/my.cnf`, `/etc/my.cnf` o `/etc/mysql/mysql.conf.d/mysqld.cnf`.
  - Para verificar el orden de búsqueda desde la terminal:

```
mysqld --help --verbose | grep -A 1 "Default options"
```

- **En Windows:**
  - Archivo: `my.ini` (habitualmente ubicado en `C:\ProgramData\MySQL\MySQL Server X.Y\my.ini` o en el directorio raíz de la instalación).
- **Para verificar directorios desde SQL:**

```
SHOW VARIABLES LIKE 'basedir';
SHOW VARIABLES LIKE 'datadir';
```

5.2 Indicar el cambio que debe realizarse en dicho archivo

Abrir el archivo de configuración (`my.cnf` o `my.ini`) y, bajo la sección del servidor `[mysqld]` (o `[mariadb]`), definir el parámetro:

```
[mysqld]
max_connections = 120
```

5.3 Consultar mediante SQL el valor activo de `max_connections`

Ejecutar en el cliente SQL de MySQL/MariaDB:

```
-- Opción 1: Consulta estándar con SHOW VARIABLES
SHOW VARIABLES LIKE 'max_connections';

-- Opción 2: Consulta directa a la variable global del sistema
SELECT @@GLOBAL.max_connections AS max_conexiones_activas;
```

5.4 Qué debe hacerse para que el cambio persistente se aplique si el valor activo aún no es 120

Existen dos acciones complementarias:

1. **Aplicación en caliente (Inmediata sin reiniciar):** Dado que `max_connections` es una **variable dinámica**, un usuario con privilegios administrativos (`SUPER` o `SYSTEM_VARIABLES_ADMIN`) puede

modificarla de inmediato en tiempo de ejecución:

```
SET GLOBAL max_connections = 120;
```

(En MySQL 8.0+ se puede ejecutar `SET PERSIST max_connections = 120;`, lo que aplica el cambio en caliente y lo guarda en el archivo `mysqld-auto.cnf` para que persista automáticamente tras reinicios).

2. **Asegurar la persistencia definitiva:** Si no se cuenta con `SET PERSIST`, se debe editar físicamente el archivo `my.cnf` / `my.ini` con la directiva `max_connections = 120` bajo `[mysqld]`. Así, cuando el servidor se reinicie en el futuro, levantará con este nuevo valor.

5.5 Si el cambio requiere reiniciar el servicio, cuándo corresponde hacerlo

- **¿Requiere reinicio obligatorio?**
  No de forma inmediata, porque `max_connections` es una variable dinámica y tiene efecto instantáneo en memoria al ejecutar `SET GLOBAL max_connections = 120;`.
- **¿Cuándo corresponde reiniciar el servicio?**
  1. Si las políticas de seguridad o de la empresa exigen que la configuración esté validada únicamente mediante el archivo físico (`my.cnf`/`my.ini`) sin comandos temporales en memoria.
  2. En dicho caso, el reinicio debe realizarse **estrictamente dentro de una Ventana de Mantenimiento (*Maintenance Window*)**, fuera del horario laboral o en momentos de baja concurrencia (típicamente de madrugada o fines de semana), con notificación previa a los usuarios para no interrumpir conexiones activas ni cortar transacciones en curso.
  3. En entornos de **Alta Disponibilidad / Clúster** (como MariaDB Galera Cluster o Replicación Primario/Secundario), el reinicio se realiza de forma **rotativa nodo por nodo (*Rolling Restart*)**, esperando que cada nodo se sincronice completamente antes de reiniciar el siguiente, garantizando servicio ininterrumpido (*Zero Downtime*).